

Chapter 8: Implementation Support

1. Introduction

- **Interactive systems** need **programming tools** to convert design ideas into **working software**.
- These tools work in layers, from **low-level hardware commands** to **high-level interface objects**.

Core Problem:

At the lowest level (lower-level programming)

- Programmer directly handles:
 - Mouse events
 - Keyboard events
 - Pixel drawing
- This is:
 - **Time-consuming + Hard to manage + Error-prone**

Higher-level systems solve this by hiding low-level details.

Interaction Techniques

- **Interaction technique = combination of input + output behavior.**
- **Hardware separates:**
 - Input devices (mouse, keyboard)
 - Output devices (screen)
- But **software combines them into one interaction object.**
- **Eg:** Mouse movement + cursor movement feel like one thing to the user.

1.1 Layers of Abstraction (Bottom to Top)

Layer	What It Provides	Example
-------	------------------	---------

Hardware / Device Drivers	Direct communication with hardware (read events, write pixels)	Mouse, Keyboard, Display
Windowing System	- Device independence + multi-application management - Allows multiple applications to run in separate windows.	X Window System (X11)
Toolkit	Ready-made interaction objects (widgets/gadgets)	XView, OSF/Motif, AWT
UIMS / UIDS	Separation of application logic and interface presentation	Seeheim model, MVC, PAC

[UIMS → user interface management system]

2. Elements of Windowing Systems (Section 8.2)

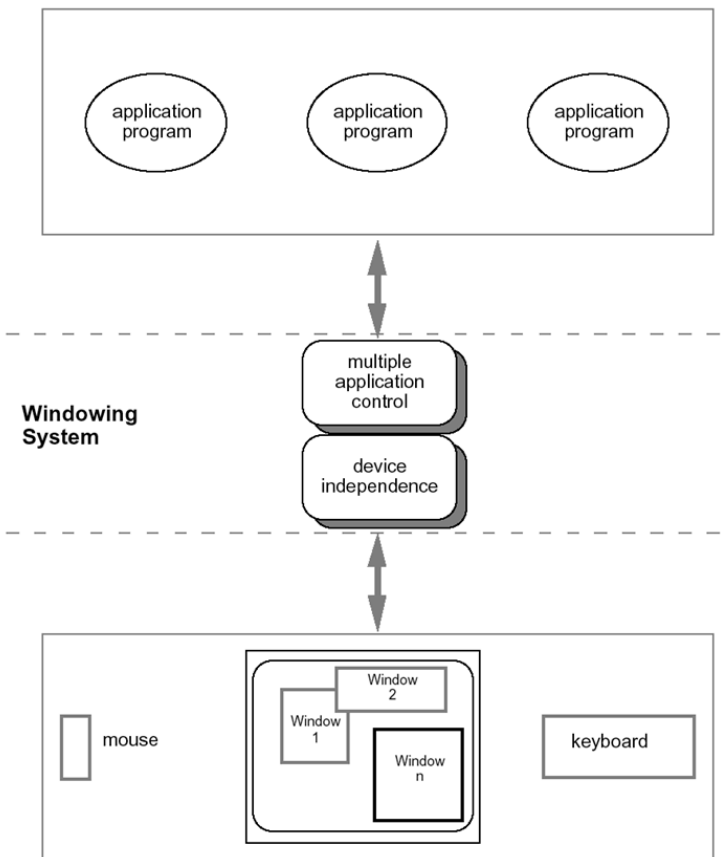
Windowing systems sit between:

- **Hardware**
- **Application programs**

They provide two major capabilities.

- **Device Independence:**
 - Applications use abstract commands instead of hardware-specific commands.
 - Same program can work on different machines. ⇒ **portable**
- **Multiple Application Management:**
 - Many applications can run simultaneously.
 - Each application gets its own window.

- Resource sharing



2.1 Key Concepts

> Abstract Terminal:

- Generic interface between applications and hardware.
- Translates **common commands into hardware-specific operations**.
- Only **one device driver is needed** per hardware device.

> Imaging Model:

- Generic language used to describe screen content.
- Represents graphic on screen [text, shapes, imgs]

> Device Driver:

- One translation program written per hardware device that allows any application program to access it.

- Translation software for hardware devices.
- Allows applications to communicate with hardware.

2.2 Examples of Imaging Models

Model	Description	Use / Notes
Pixels	> screen represented as tiny dots > pixels can be turned on/off or colored	Common in PCs, used by X Window System
GKS (Graphical Kernel System)	> Screen made from connected graphic segments.	International standard
PHIGS	> Extension of GKS. > Uses editable graphic segments.	International standard, extension of GKS
PostScript	> Uses paths and shapes filled with colors/patterns.	Developed by Adobe

Input Role of Imaging Models:

Originally used only for output, but later used for input too.

- > **Pixel Model:** Mouse input identified using pixel coordinates.
- > **PHIGS / PostScript:** Input linked to application objects directly.

2.3 Architectures of Windowing Systems

Bass and Coutaz identify 3 possible architectures (all assume device drivers are separate from applications)

- **Architecture 1 — Replicated within each application**
 - Every application manages its own coordination.
 - **PROBLEMS:**

- Synchronization conflicts
 - Poor portability
 - More programmer work
- **Architecture 2 — Within the OS kernel:**
 - Window management handled by operating system kernel.
 - **Problem:** Applications depend heavily on specific OS.
- **Architecture 3 — Client-Server (separate application)**
 - Window management is a separate application.
 - **Adv:**
 - Most portable
 - Generic interface
 - Works across operating systems
 - Most widely used architecture

Important Note: In practice, the divide among these architectures is not clear — real systems may blend features from all three.

2.4 The X Window System (X11) — Classic Client-Server Example

Developed at MIT

Main Features

- Client-server architecture
- Pixel-based imaging model
- Uses pointing devices

What makes X unique/popular: Based on a network protocol (**X Protocol**) that **clearly defines server-client communication**. Can be implemented on different computers and OSes, making it **device independent**. **Client and server** can be on **different machines**.

X Server Tasks:

- **Allows (or denies) access** to display from multiple client applications
- **executes display requests** from clients
- **Demultiplexes** the stream of **physical input events** and passes them to the appropriate client ⇒ **event distribution**
- **stores shared resources** like fonts— clients access these by ID numbers

Window Manager (separate client):

- Separate client in X11.
- Resolves conflicts (input/output requests) between windows.
- Multiple window managers can be used in X.
- **Policy decisions:**
 - **Input focus:** **explicit** (user **nominates active window**) vs **implicit** (active window = **window under pointer**)
 - **Window layout:** **overlapping** windows (can **cover each other**) vs **tiling** (non-overlapping)

Client Subwindows:

- Applications can create **subwindows inside** their **main window**
- Each application manages its own internal layout.

ICCCM (Inter-Client Communication Conventions Manual):

Produced by X Consortium to aid design of window managers.

- **PURPOSE is to provide rules for:**

- Data transfer between clients
- Input focus selection
- Window layout management

3. Programming the Application (Section 8.3)

Interactive applications are **user-driven** — the **action taken depends on input from user**. Two fundamental programming paradigms control **how events are processed**. The windowing system does NOT determine which paradigm to use.

3.1 Paradigm 1: Read-Evaluation Loop (Internal Control)

- **Application itself controls event processing**
- The **main control loop resides within the application** program itself

How it works:

- Server sends **user inputs as structured events** to the client
- Application **reads each event** and **determines application-specific behavior**
- Application has **COMPLETE CONTROL over event processing**

Pseudocode Structure:

```
repeat
  read-event(myevent)
  case myevent.type
    type_1 : do type_1 processing
    type_2 : do type_2 processing
    ...
    type_n : do type_n processing
  end case
```

1. Read event
2. Check event type
3. Execute corresponding action
4. Repeat forever

end repeat

ADVANTAGES	DISADVANTAGES
Complete control over every event	Must handle EVERY possible event — very complex
Pre-emptive dialogs are straightforward (nested loops)	Higher programmer burden
Natural fit for modal dialog boxes	Less separation between UI and application logic

3.2 Paradigm 2: Notification-Based (External Control)

- A **centralized notifier** handles event dispatching.

How it works:

- Application **registers** which **events** are **of interest** to it
- For each event, application **declares a callback procedure** before handing control to the notifier
- When **notifier receives a relevant event**, it **passes control to the registered callback**
- **Callback returns control to notifier**, either to **continue or terminate**
[**CALLBACK**: function executed when specific event occurs]

Example: XView quit.c program

- Three objects created: outermost FRAME, PANEL within it, and a PANEL_BUTTON in the panel
- `xv_create()` registers objects with the XView notifier
- `PANEL_NOTIFY_PROC` specifies the callback procedure (`quit`) for button selection
- `xv_main_loop()` hands control to the notifier
- When button selected, notifier calls `quit()` which destroys the frame

ADVANTAGES	DISADVANTAGES
Relieves app of processing every possible event	Control flow out of programmer's hands
Cleaner separation of concerns	Pre-emptive dialogs are DIFFICULT — all callbacks must be modified
Natural fit for non-modal interfaces	Less suitable for modal interactions

[what is **modal dialog**?]

a window overlaid on an application's main content that requires the user to interact with it before they can return to the primary workflow. It temporarily disables the background content, forcing the user's attention onto a specific decision, warning, or task.

3.3 Design Focus: Going with the Grain

Key Concept — 'Grain': Each programming style naturally supports certain interfaces.

- **Notification-based:** naturally produces **non-modal (non-pre-emptive)** dialogs; modal dialogs possible but much harder
- **Event-loop-based:** naturally produces **modal (pre-emptive)** dialogs; non-pre-emptive possible but harder

Practical advice: Choose the paradigm that matches the required interface behavior

4. Using Toolkits (Section 8.4)

Toolkits **combine Input & Output** into single interaction objects.

4.1 The Mouse as an Input-Output Fusion

- Hardware separates:
 - Mouse input
 - Cursor output
- But users see them as one object. When this link breaks, user frustration results.
- Toolkits create this illusion programmatically.

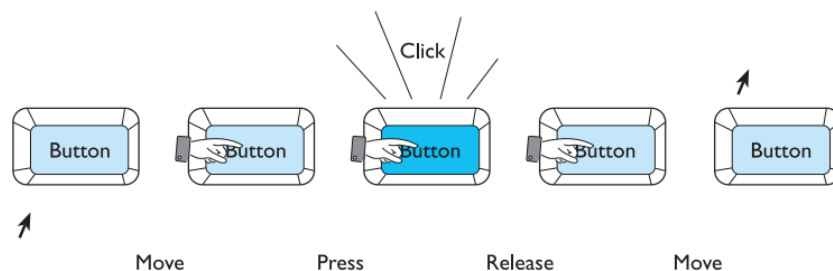
4.2 What is a Toolkit?

- Toolkit = **collection of ready-made interaction objects.**
- Also called:
 - **Widgets**
 - **Gadgets**
 - **Interaction techniques**

These objects have **predefined behaviors** that come for free **without additional programming effort**

Button Behavior Example (Figure 8.8):

- Mouse moves over button → cursor changes to finger (affordance cue)
- Mouse button pressed → button highlighted + audible click (immediate feedback)
- Mouse button released → button unhighlighted
- Mouse moved off button → cursor returns to arrow shape



Toolkit Examples by Platform:

- X Window System: OSF/Motif, XView
- Macintosh: Macintosh Toolbox
- Microsoft Windows: Software Development Toolkit (SDK)
- Java: Abstract Windowing Toolkit (AWT)

4.3 Flexibility of Interaction Objects

- Objects can be **tailored via parameters** (e.g., label text on a button)
- **Complex** interaction **objects built from simpler ones**
- Entire application = **collection of interaction objects** whose **combined behavior describes application semantics**
- **Programming with toolkits** is suited to the **notification-based paradigm**

Higher-level abstraction: The programmer thinks of events at a higher level (e.g., 'selection event' on a button, rather than 'release of left mouse button'). The **toolkit handles the low-level details**.

4.4 Look and Feel

Consistency of behavior across applications using same toolkit

- > all buttons behave similarly

Style Guides:

- > **Extra design rules** to maintain consistency.

4.5 Object-Oriented Nature of Toolkits

- **Instantiation:**
 - same class can create many objects.
 - Eg; many buttons from one button class
- **Inheritance:**
 - complex classes inherit features from simpler classes.

Class Hierarchy in XView (Figure 8.9):

Single Inheritance example — Window is the root class:

- Window → Frame, Tty, (Openwin), Scrollbar, Icon
- (Openwin) → Text, Canvas
- Canvas → Panel

Single Inheritance: Each class **inherits from exactly one parent** class.

Multiple Inheritance: New classes **inherit from more than one parent** (more complex)

Instance Attributes:

Class templates provide default values.

> Attributes that **can be altered per instance** are called **instance attributes**.

Java AWT Design Focus:

- Java toolkit for windowed applications is called the Abstract Windowing Toolkit (AWT)
- Maps interface objects (buttons, menus, dialogs) to corresponding Java classes
- Programmer uses classes directly or subclasses them (specializes behavior)
- AWT 1.0: subclass button to specify press behavior
- AWT 1.1+: use callback-like approach, registering Java objects instead of functions
- Notification-based; subclassing means new widgets can easily be added

Important: Object orientation is a programming paradigm, NOT a language requirement. The XView quit.c example is written in C (not OO), but uses OO-structured toolkits.

5. User Interface Management Systems

[UIMS/UIDS] (Section 8.5)

- Toolkits still have limitations.
- UIMS adds another higher layer.

5.1 Limitations of Toolkits

- Provide only a **limited range of interaction objects**
- **Expensive** to create
- **Very difficult to use by non-programmers**
- Even expert **programmers struggle to produce predictably usable interfaces**
- None of the preceding programming mechanisms are appropriate for non-expert programmers

5.2 What is UIMS / UIDS (User Interface Development Systems)

- Programming and design techniques **adding another level of services beyond toolkits.**
- Provides:
 - Better design support
 - Better separation of interface and application logic

Main Concerns of a UIMS:

- **conceptual architecture:** separates application semantics and presentation
- **Implementation techniques:** Maintains connection between separated components.

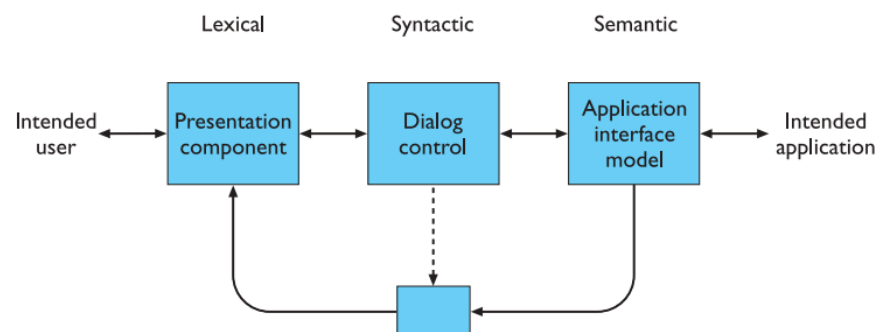
- **Runtime Support:** Helps manage and evaluate interactive systems.

5.3 Why Separation of Application and Presentation?

Reason	Explanation
Portability	Same app works on different systems.
Reusability	Components reused in other projects, cutting development costs
Multiple Interfaces	Same application can have different interfaces.
Customization	UI changes without modifying application logic

5.4 UIMS as a Conceptual Architecture – The Seeheim Model

Provides three major components: **Application, Presentation, and Dialog Control**



Three Logical Components (Seeheim Model):

- **Presentation Component:** Responsible for the **interface appearance** — what output and input is available to the user / **IO handling** (**Lexical** layer)
- **Dialog Control Component:** handles **communication between application and presentation**. (**Syntactic** layer)

- **Application Interface Model:** The view of the [application semantics](#) provided as the interface. (**Semantic** layer)

Layers map to compiler design: [Lexical \(presentation\)](#) → [Syntactic \(dialog\)](#) → [Semantic \(application\)](#) — familiar from compiler design theory.

Bypass / Switch (lower box in Seeheim model):

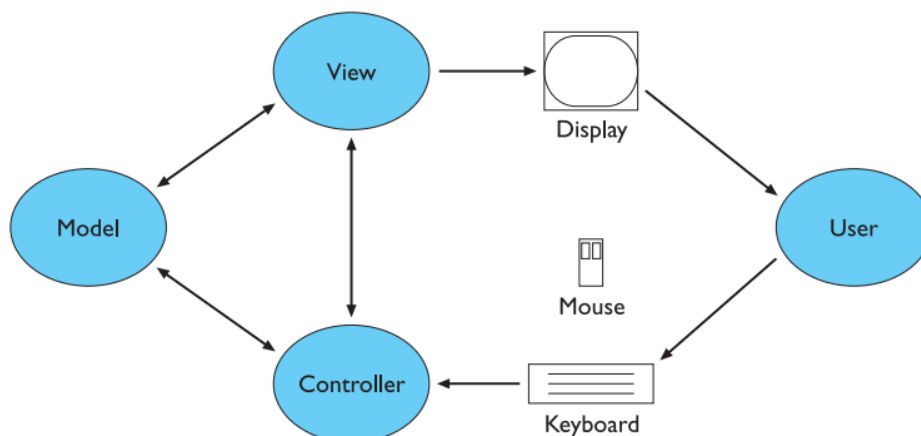
- › Allows [application to directly access presentation](#) for [fast feedback / efficiency](#)
- › Dialog controls WHEN access is allowed but does not mediate the presentation content.

Criticism of Seeheim Model:

- [did not provide direction for future UIMS](#) / did not guide future designs
- [Did not separate logical concerns from implementation concerns](#) (e.g., the bypass box)
- [Too restrictive](#)
- Led to the development of the [Arch-Slinky model](#) (more layers, more fluid mapping)

5.5 Model-View-Controller (MVC) Paradigm

Builds link between application semantics and presentation in units via the MVC triad.



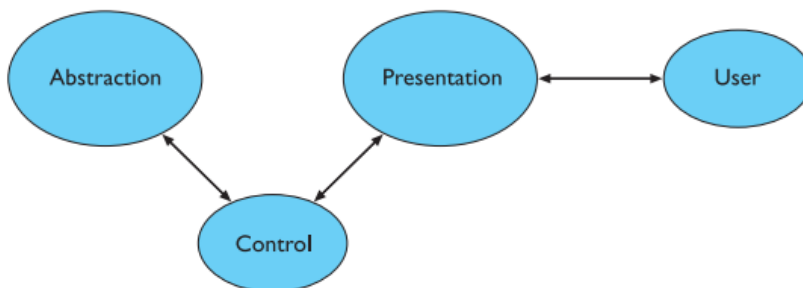
- **Model:** Represents the **application semantics**. Very general — must portray any application semantics. **One model can connect to multiple views / controllers** (same semantics, different input-output techniques).
- **View:** Manages the **graphical and/or textual output** of the application.
- **Controller:** **Manages the input**.

Each view-controller pair is associated with only one model.

5.6 Presentation-Abstraction-Control (PAC) Model

Based on a collection of triads

- **Abstraction:** **application semantics** (equivalent to Model in MVC).
- **Presentation:** **Handles both input AND output** combined (contrast with *MVC which separates them*).
- **Control:** Explicit component **managing dialog** and **consistency between abstraction and presentation**.



Three Key Differences — PAC vs MVC:

Aspect	MVC	PAC
Input/Output	Separated (View = output, Controller = input)	Combined in one Presentation component
Consistency control	Not assigned to any one component — programmer's responsibility	Explicit Control component ensures consistency

Environment coupling	Linked to Smalltalk	Independent conceptual architecture / model
----------------------	---------------------	---

5.7 Implementation Techniques for Dialog Control

1) Menu Networks:

- Dialog represented as **connected menus**
- Communication modeled as a **network of menus/submenus**.
- **Menu = all possible user inputs at any one time.**
- Eg; HyperCard

2) Grammar Notations:

- Dialog treated like formal grammar.
- **grammar of actions/responses**, described by **formal context-free grammar**
- Good for command based systems
- Poor for graphical interaction
- Hard to model feedback

3) State Transition Diagrams:

- **Graphical** dialog representation.
- Difficult communication modeling
- Difficult to link dialog events with presentation/application events

4) Event Languages

- Uses **production rules**.
- Support event direction & some semantic feedback
- Harder to model overall dialog flow.

5) Declarative languages

- Describe **WHAT should happen, not HOW**
- Focus is on relationship between values
- Describe relationships, not procedures.

6) Constraints

- Special **type of declarative language.**
- **Must always be maintained**
- e.g., piggy bank display value = actual value inside

7) Graphical specification (Programming by Demonstration)

- **Programmer builds interface visually**
- Dialog specified graphically **in terms of the actual presentation language**
- **Major Advantage:** **Non-programmers can design dialogs.**

ALV (Abstraction-Link-View)

- Created by: Ralph Hill
- **Same distinctions as PAC.**
- Link component = **collection of two-way constraints between abstraction and view.**
- Constraints → **dependencies that must always be maintained.**
- **Abstraction** → application logic/semantics
- **View** → presentation

Spectrum of Dialog Control:

- **Internal to application** (read-eval loop) → **app must know presentation issues** (less generic)
- **External/independent dialog component** → independent but can be **complicated** (must represent info from both app and presentation)

- **Presentation control** (graphical specification) → **simple** but **restricted**; graphical language less expressive than programming languages

Garnet System

- Combines declarative constraints language with graphical specification technique.

6. Chapter Summary

6.1 Programming Paradigm Comparison

Feature	Read-Evaluation Loop	Notification-Based
Control location	Internal (within application)	External (centralized notifier)
Pre-emptive dialogs	Easy (nested loops)	Difficult (all callbacks must be modified)
Natural dialog style	Modal (pre-emptive)	Non-modal (non-pre-emptive)
Example	Macintosh programming, MacApp	XView, X Toolkit (Xt)

6.2 Key Definitions Glossary

Term	Definition
Abstract Terminal	Generic interface that translates a common command language to specific hardware — enables device independence
Imaging Model	The fixed generic language of an abstract terminal for describing screen content
Device Driver	Translation program written once per hardware device; allows any application to access it

Interaction Technique	Term indicating the intimate relationship between input and output
Widget / Gadget	A ready-made interaction object provided by a toolkit
Look and Feel	Consistency of behavior for interaction objects within a toolkit
Callback	Application procedure registered with a notifier to be invoked when a specific event occurs
Notifier	Centralized component in notification-based programming that dispatches events to callbacks
UIMS	User Interface Management Systems — tools adding services beyond toolkits, separating app semantics from presentation
UIDS	User Interface Development Systems — preferred term emphasizing design support over runtime management
Dialog Control	The role of managing communication between application and presentation
Seeheim Model	1985 conceptual UIMS architecture with Presentation, Dialog Control, and Application Interface components
MVC	Model-View-Controller: OO architecture in Smalltalk separating application (Model), output (View), and input (Controller)
PAC	Presentation-Abstraction-Control: architecture combining I/O in Presentation, app in Abstraction, explicit Control component
ALV	Abstraction-Link-View by Hill — PAC-like, uses two-way constraints in the Link component
ICCCM	Inter-Client Communication Conventions Manual — X Consortium conventions for window manager policy